

Microservicio en GraphQL para la gestión de usuarios

1. Análisis del proyecto integrador

- Identificación clara del endpoint

Endpoints ha consumir:

1. Para obtener el token de autenticación:

POST auth/login

2. Para verificación en dos factores:

POST auth/verify-2fa

3. Para obtener la lista de usuarios:

GET auth/users

- Justificación de la elección del endpoint

Se eligió este endpoint auth/login porque:

- En el proyecto de Testimonios se manejan roles, uno de esos roles es el ADMIN. El ADMIN es aquel que puede ver la lista de los usuarios pero para ello debo logearse.
- Sin este endpoint, no se podría acceder a las funcionalidades protegidas.
- Permite demostrar el manejo de credenciales sensibles de forma segura.

Se eligió este endpoint auth/verify-2fa porque:

- Especialmente relevante para roles como ADMIN y CURADOR que requieren mayor seguridad.
- Demuestra cómo proteger operaciones sensibles.
- Ejemplo de arquitectura en capas de seguridad

Se eligió este endpoint auth/users porque:

- Esencial para la gestión de usuarios.

- Descripción técnica del endpoint (estructura, método, formato de datos)

Característica	Descripción
Método	POST auth/login, POST auth/verify-2fa, GET auth/users
URL	auth/login, auth/verify-2fa, auth/users
Formato de datos	GraphQL
Formatos de respuesta	JSON
Autenticación	Token en headers "Authorization: Bearer <token>"

2. Diseño del Microservicio

- Objetivo del microservicio claramente definido

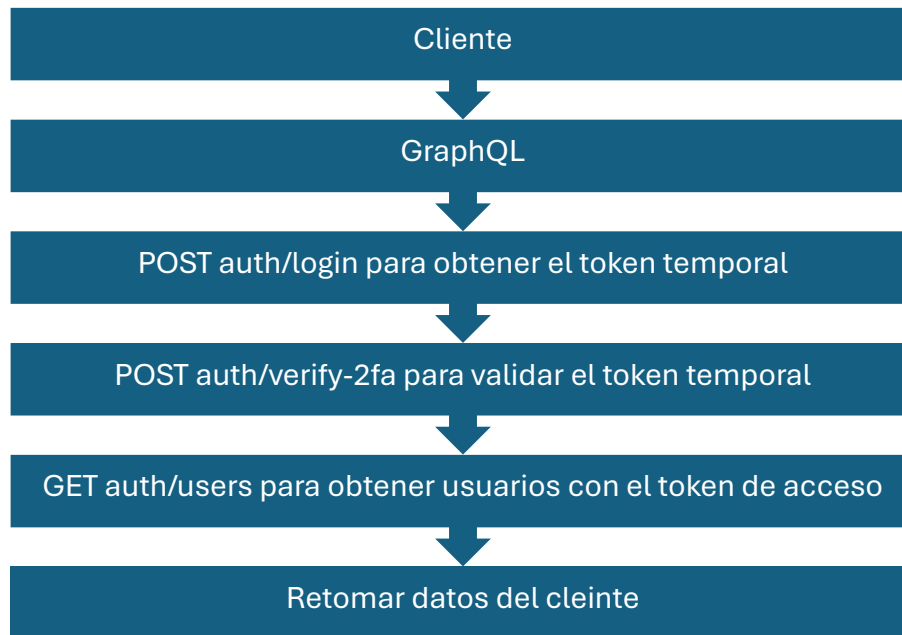
Objetivo: Construir un microservicio GraphQL que permita a los administradores de la página web de Testimonios consultar usuarios desde la API auth, manejando la autenticación, verificación en dos factores y consultas a través de este endpoint.

- Elección y justificación de la tecnología de comunicación

Tecnología elegida: GraphQL

- Reduce el número de llamadas necesarias.
- Permite al cliente solicitar solo los datos que necesita.
- Facilita la evolución del API sin versionado.

- Diagrama del flujo de integración



3. Implementación Técnica

Estructura del proyecto:

/AndresMenchaca_2daEvaluacionBackend

```
-- src/  
|  |-- types/  
|  |  |-- graphql.d.ts  
|  |-- server.ts  
|  |-- resolvers.ts  
|  |-- schema.graphql  
|  |-- apiClient.ts  
|  |-- config.ts  
-- .env  
-- package.json  
-- tsconfig.json  
-- README.md
```

- Desarrollo del microservicio funcional

• server.ts

```
import { createServer } from "node:http";  
import { createYoga } from "graphql-yoga";  
import { makeExecutableSchema } from "@graphql-tools/schema";
```

```

import { readFileSync } from "fs";
import { join } from "path";
import resolvers from "../resolvers";
import { config } from "../config";

const typeDefs = readFileSync(join(__dirname, "schema.graphql"), "utf-8");

const schema = makeExecutableSchema({
  typeDefs,
  resolvers,
});

const yoga = createYoga({
  schema,
  context: ({ request }) => ({
    authHeader: request.headers.get("authorization") || "",
  }),
});

const server = createServer(yoga);

server.listen(config.port, () => {
  console.log(`Servidor corriendo en el puerto ${config.port}`);
});

```

- Consumo correcto del endpoint externo

• apiClient.ts

```

import axios, { AxiosError } from "axios";
import { config } from "../config";

interface ApiError {
  message: string;
  [key: string]: any;
}

interface ApiResponse {
  data?: any;
  message?: string;
}

const api = axios.create({
  baseURL: config.apiUrl,
  timeout: 5000,
});

function isApiError(error: any): error is { response: { data: ApiError } } {

```

```

    return error?.response?.data?.message !== undefined;
}

const apiClient = {
  login: async (email: string, password: string) => {
    try {
      const response = await api.post("/auth/login", { email, password });
      return response.data;
    } catch (error) {
      if (isApiError(error)) {
        throw new Error(error.response.data.message);
      }
      throw new Error("Error en el login");
    }
  },

  verify2FA: async (token: string, tempToken: string) => {
    try {
      const response = await api.post(
        "/auth/verify-2fa",
        { token },
        {
          headers: { Authorization: `Bearer ${tempToken}` },
        },
      );
      return response.data;
    } catch (error) {
      if (isApiError(error)) {
        throw new Error(error.response.data.message);
      }
      throw new Error("Error en la verificación 2FA");
    }
  },

  getUsers: async (accessToken: string) => {
    try {
      const response = await api.get("/auth/users", {
        headers: { Authorization: `Bearer ${accessToken}` },
      });
      return response.data;
    } catch (error) {
      if (isApiError(error)) {
        throw new Error(error.response.data.message);
      }
      throw new Error("Error obteniendo usuarios");
    }
  }
};

```

```

    }
  },
};

```

```
export default apiClient;
```

- **Procesamiento y transformación de los datos**

- **resolvers.ts**

```
import apiClient from "../apiClient";
```

```
interface AuthResponse {
  accessToken?: string;
  refreshToken?: string;
  user?: {
    id: number;
    email: string;
    nombre?: string;
    biografia?: string;
    role: number;
    two_factor_enabled: boolean;
  };
  requires2FA?: boolean;
  tempToken?: string;
  message?: string;
}

```

```
interface Context {
  authHeader?: string;
}

```

```
const resolvers = {
  Query: {
    users: async (_, __: any, context: Context) => {
      try {
        if (!context.authHeader) {
          throw new Error("Acceso no autorizado: Token requerido");
        }

        const users = await apiClient.getUsers(
          context.authHeader.replace("Bearer ", ""),
        );
        return users.map((user: any) => ({
          id: user.id_usuario,
          email: user.email,
          nombre: user.nombre,
          biografia: user.biografia,

```

```

        role: user.id_rol,
        two_factor_enabled: user.two_factor_enabled,
        last_login: user.last_login,
    }));
} catch (error) {
    console.error("Error en resolver users:", error);
    throw new Error(
        error instanceof Error ? error.message : "Error desconocido",
    );
}
},
},
},

```

```

Mutation: {
    login: async (
        _: any,
        { email, password }: { email: string; password: string },
    ): Promise<AuthResponse> => {
        try {
            const response = await apiClient.login(email, password);

            if (response.requires2FA) {
                return {
                    requires2FA: true,
                    tempToken: response.tempToken,
                    message: response.message,
                };
            }

            return {
                accessToken: response.accessToken,
                refreshToken: response.refreshToken,
                user: response.user
                    ? {
                        id: response.user.id,
                        email: response.user.email,
                        nombre: response.user.nombre,
                        biografia: response.user.biografia,
                        role: response.user.role,
                        two_factor_enabled: response.user.two_factor_enabled,
                    }
                    : undefined,
            };
        } catch (error) {
            console.error("Error en resolver login:", error);
        }
    }
}

```

```

        throw new Error(
            error instanceof Error ? error.message : "Error en el login",
        );
    }
},

verify2FA: async (
    _: any,
    { token, tempToken }: { token: string; tempToken: string },
): Promise<AuthResponse> => {
    try {
        const response = await apiClient.verify2FA(token, tempToken);

        return {
            accessToken: response.accessToken,
            refreshToken: response.refreshToken,
            user: response.user
                ? {
                    id: response.user.id,
                    email: response.user.email,
                    nombre: response.user.nombre,
                    biografia: response.user.biografia,
                    role: response.user.role,
                    two_factor_enabled: response.user.two_factor_enabled,
                }
                : undefined,
        };
    } catch (error) {
        console.error("Error en resolver verify2FA:", error);
        throw new Error(
            error instanceof Error ? error.message : "Error en verificación 2FA",
        );
    }
},

UserRole: {
    ADMIN: 1,
    CURADOR: 2,
    INVESTIGADOR: 3,
    VISITANTE: 4,
},
};

export default resolvers;

```

- **Exposición de endpoint propio/documentado**

- **schema.graphql**

```
enum UserRole {  
  ADMIN  
  CURADOR  
  INVESTIGADOR  
  VISITANTE  
}
```

```
type User {  
  id: ID!  
  email: String!  
  nombre: String  
  biografia: String  
  role: UserRole!  
  two_factor_enabled: Boolean!  
  last_login: String  
}
```

```
type AuthResponse {  
  accessToken: String  
  refreshToken: String  
  user: User  
  requires2FA: Boolean  
  tempToken: String  
  message: String  
}
```

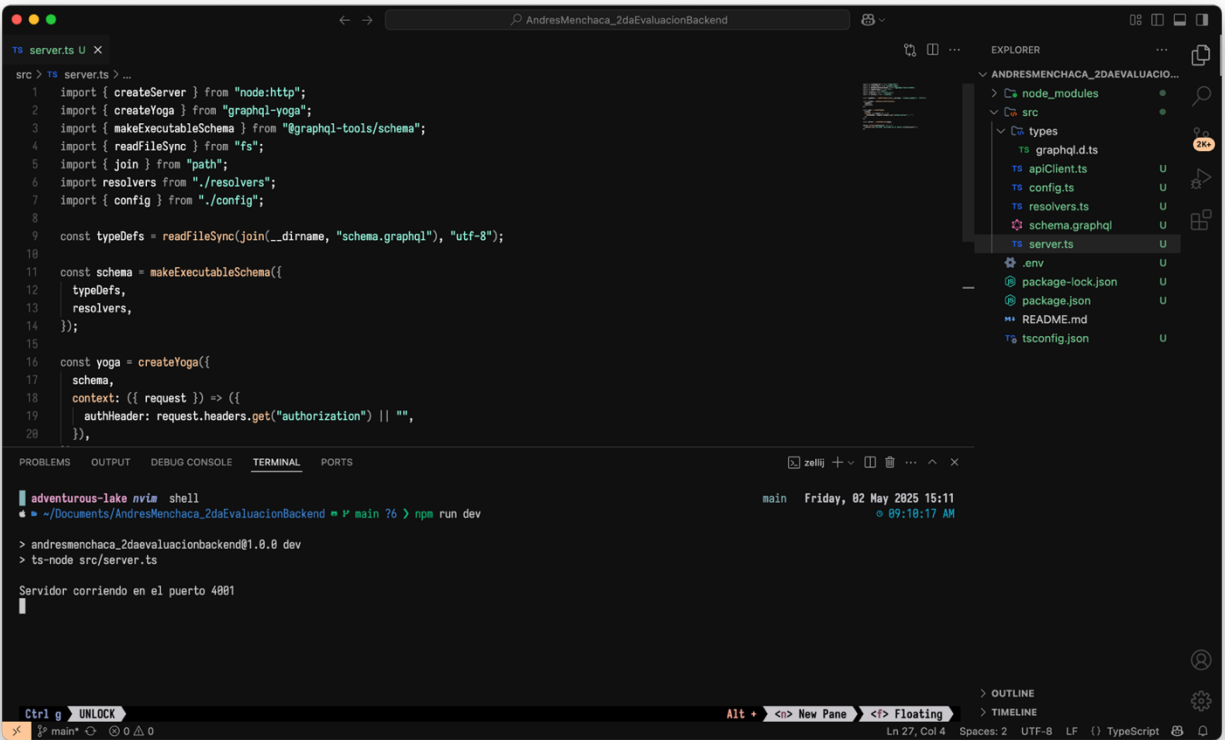
```
type Query {  
  users: [User!]!  
}
```

```
type Mutation {  
  login(email: String!, password: String!): AuthResponse!  
  verify2FA(token: String!, tempToken: String!): AuthResponse!  
}
```

4. Pruebas y Documentación

- Evidencias funcionales (capturas, video, Postman, etc.)

Para correr el microservicio: **npm run dev**



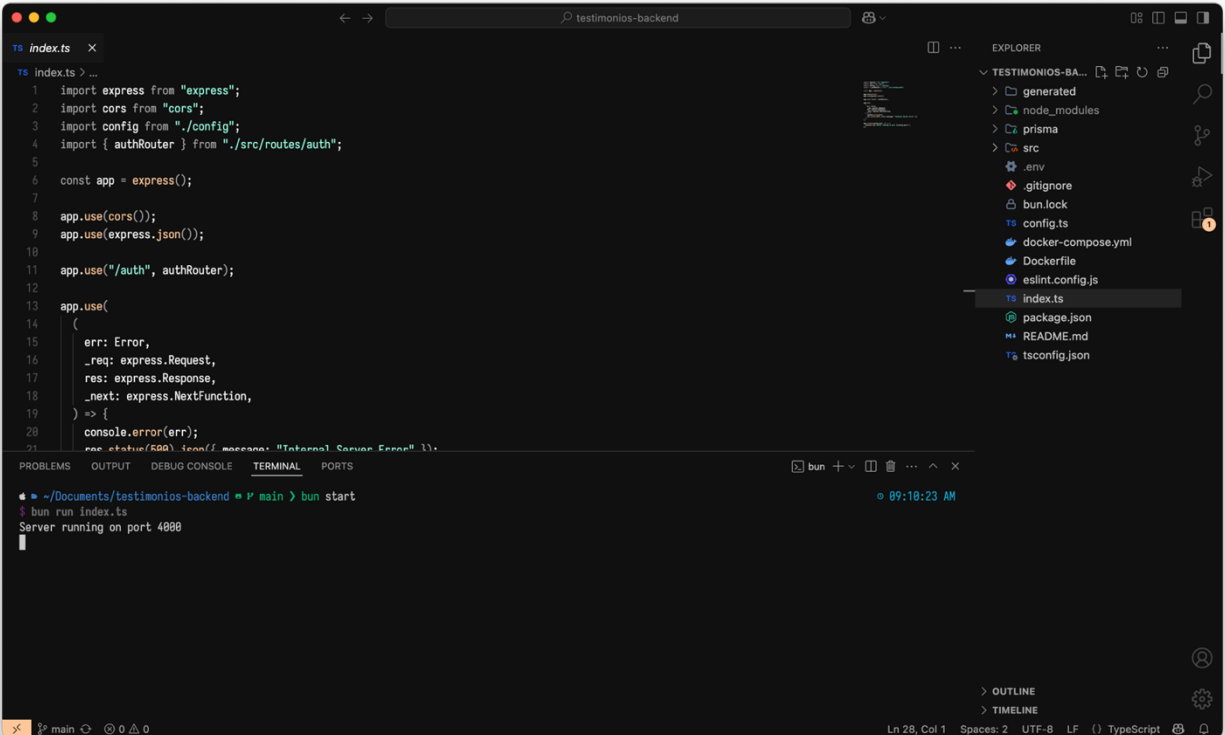
```
src > Ts server.ts > ...
1 import { createServer } from "node:http";
2 import { createYoga } from "graphql-yoga";
3 import { makeExecutableSchema } from "@graphql-tools/schema";
4 import { readFileSync } from "fs";
5 import { join } from "path";
6 import resolvers from "./resolvers";
7 import { config } from "./config";
8
9 const typeDefs = readFileSync(join(__dirname, "schema.graphql"), "utf-8");
10
11 const schema = makeExecutableSchema({
12   typeDefs,
13   resolvers,
14 });
15
16 const yoga = createYoga({
17   schema,
18   context: ({ request }) => ({
19     authHeader: request.headers.get("authorization") || "",
20   }),
21 });
22
23 const server = createServer({
24   port: 4001,
25   url: "http://localhost:4001",
26   server: yoga,
27 });
28
29 server.listen();
```

adventurous-lake nvim shell main Friday, 02 May 2025 15:11 09:10:17 AM

> andresmenchaca_2daevaluacionbackend@1.0.0 dev
> ts-node src/server.ts

Servidor corriendo en el puerto 4001

Para correr el backend: **bun start**



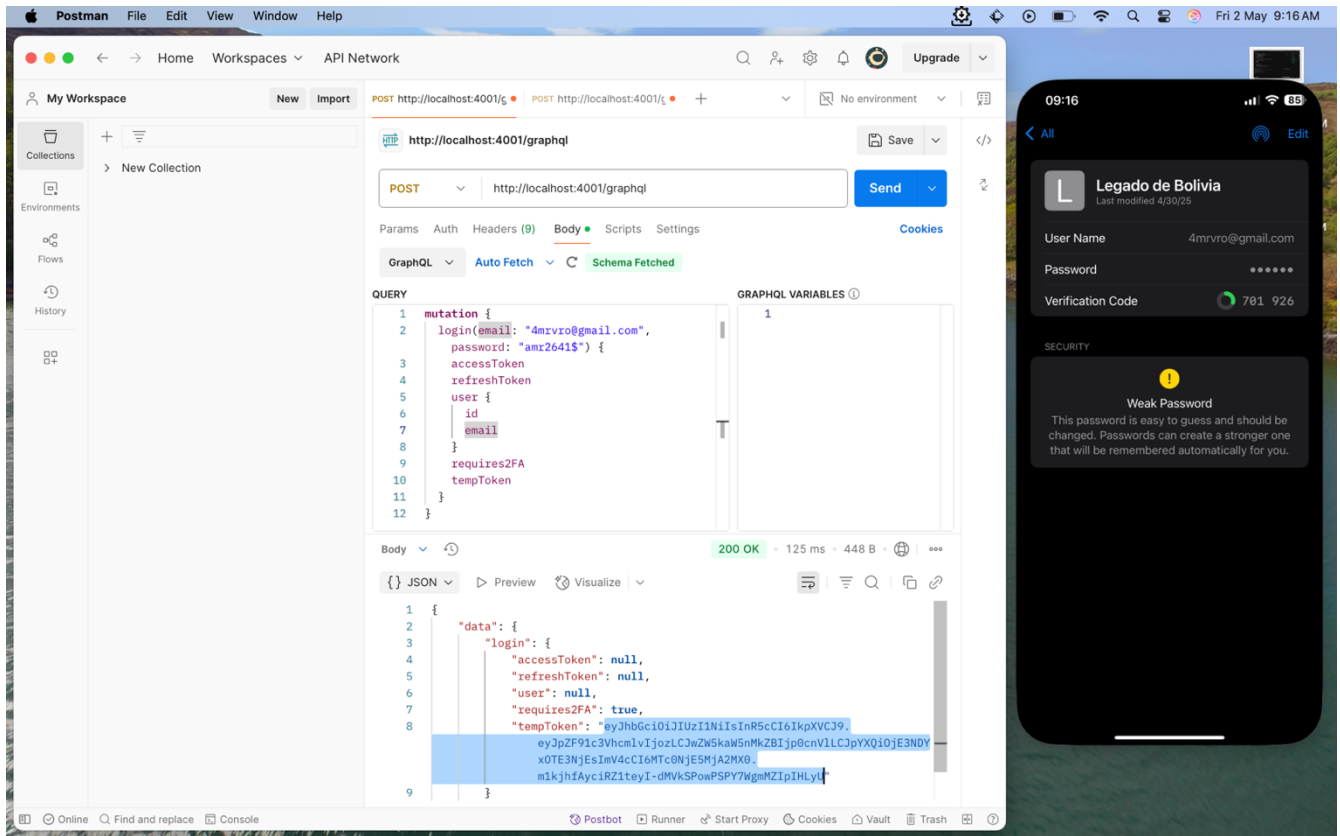
```
src > index.ts > ...
1 import express from "express";
2 import cors from "cors";
3 import config from ".config";
4 import { authRouter } from ".src/routes/auth";
5
6 const app = express();
7
8 app.use(cors());
9 app.use(express.json());
10
11 app.use("/auth", authRouter);
12
13 app.use(
14   (err, _req, res, _next) => {
15     console.error(err);
16     res.status(500).send({ message: "Internal Server Error" });
17   }
18 );
19
20 app.listen(4000, () => {
21   console.log("Server running on port 4000");
22 });
```

main Friday, 02 May 2025 15:11 09:10:23 AM

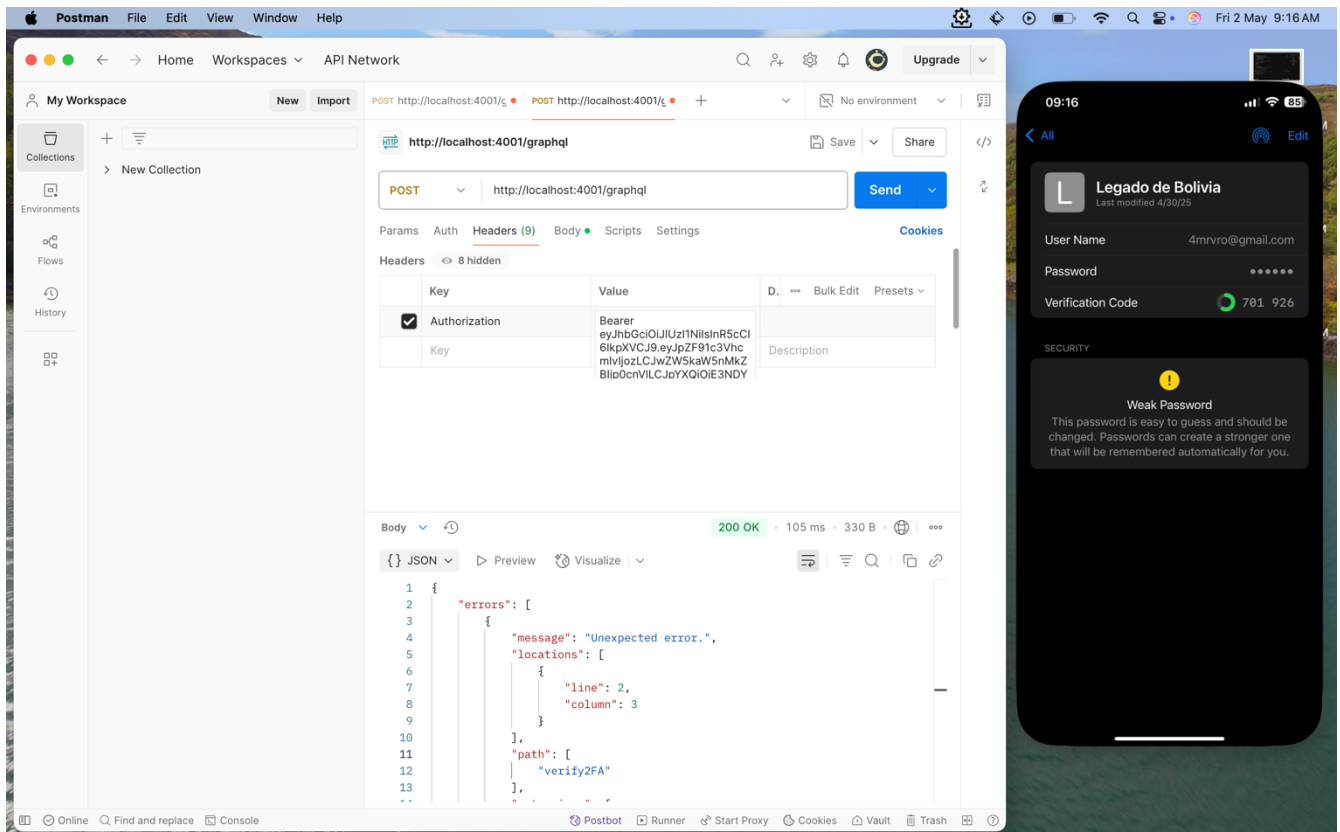
\$ bun run index.ts

Server running on port 4000

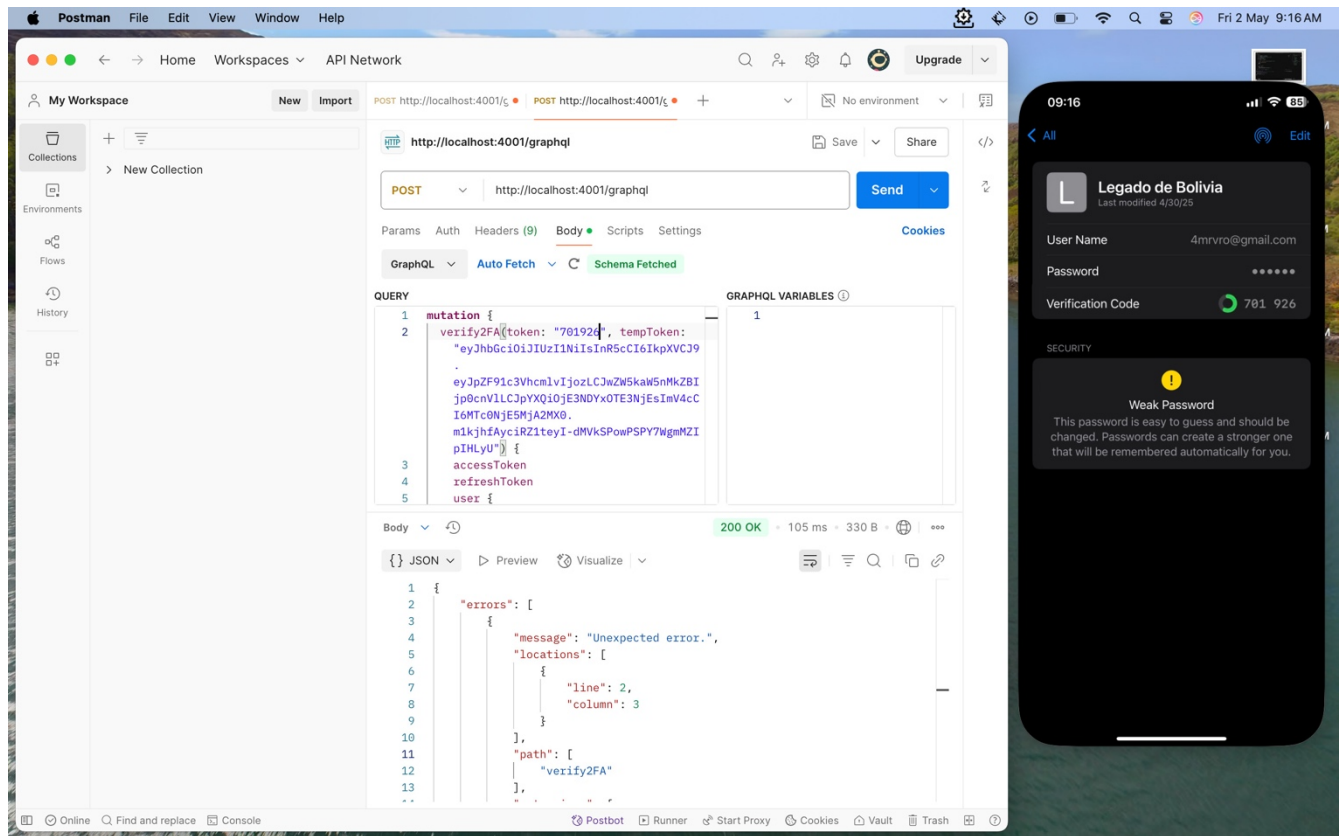
Enviarnos la petición POST para el login en <http://localhost:4001/graphql>



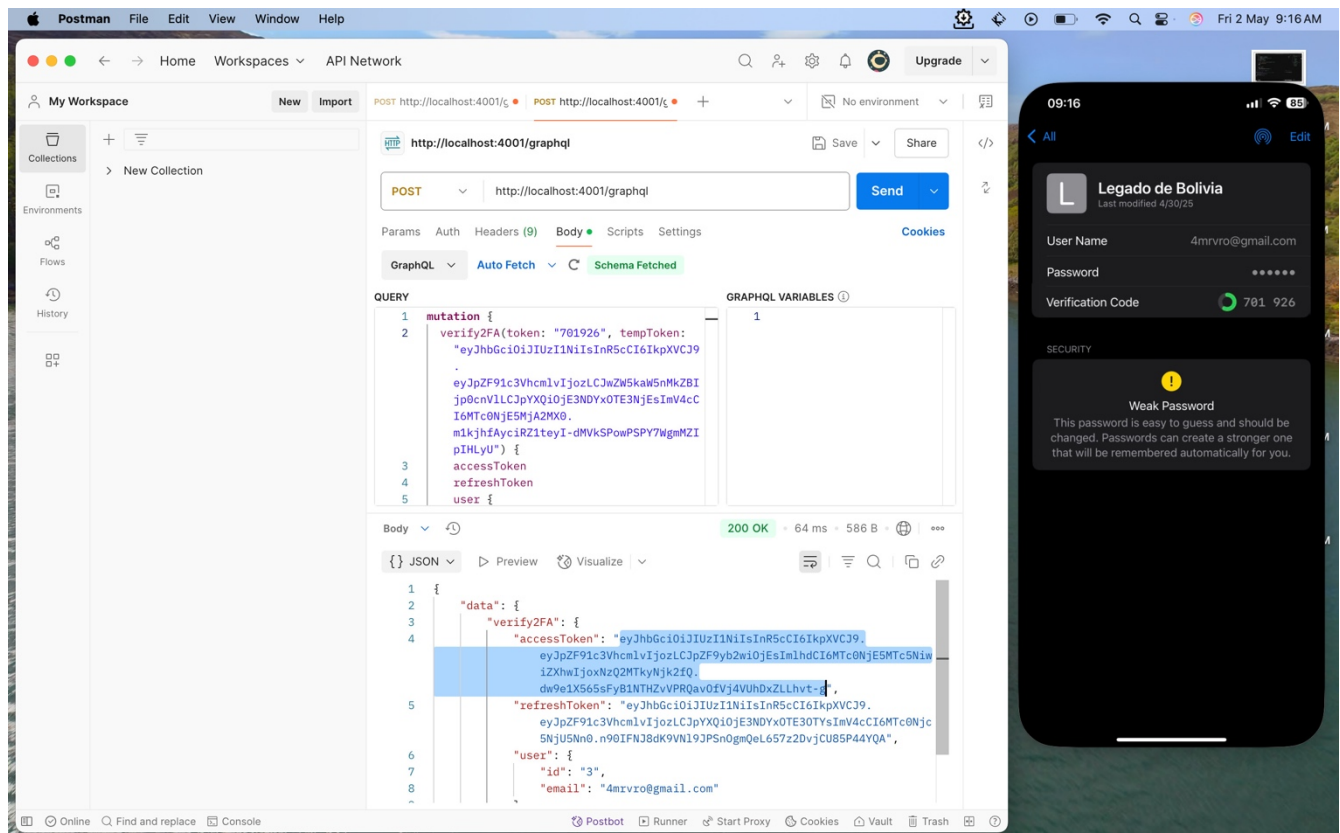
Copiamos el tempToken y lo pegamos en Header



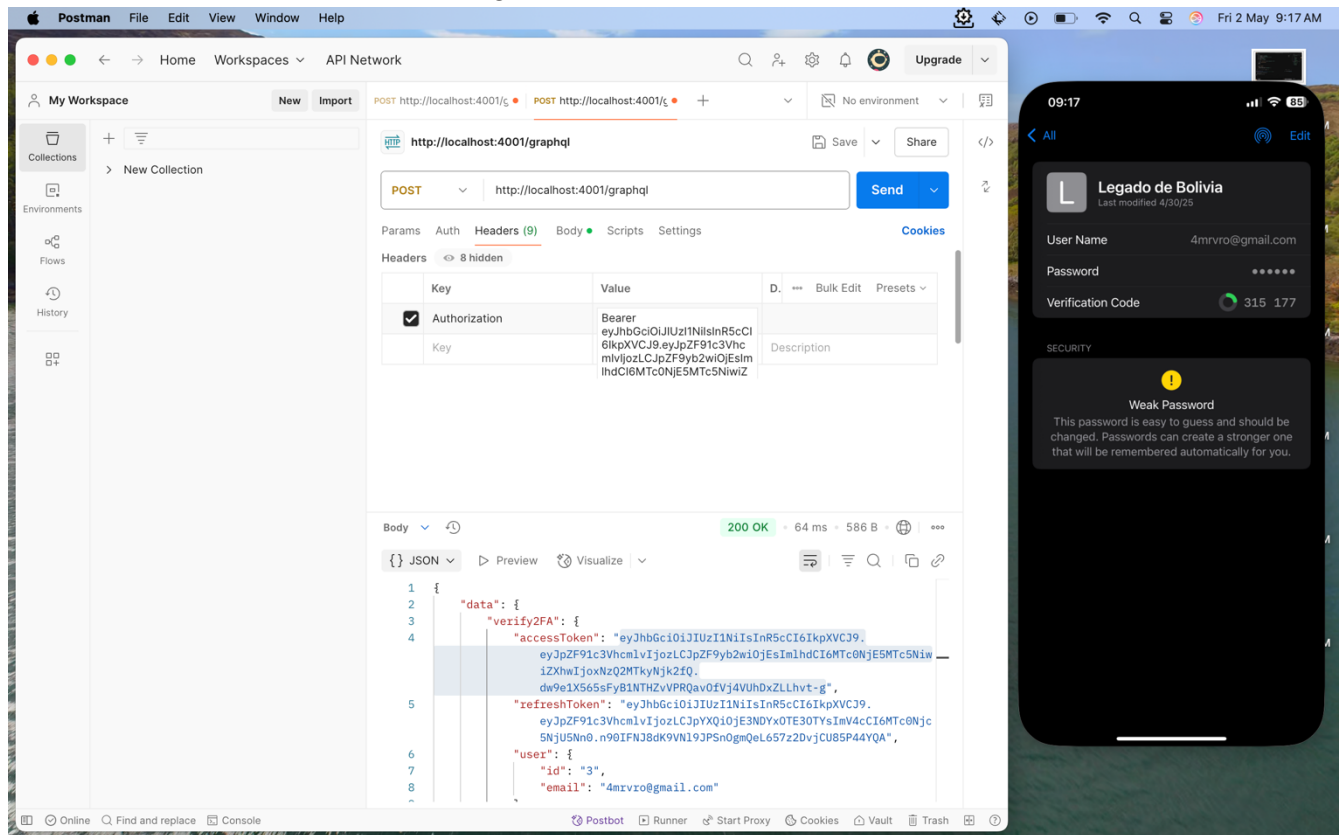
Cambiamos el Query y copiamos el código del celular y el tempToken nuevamente



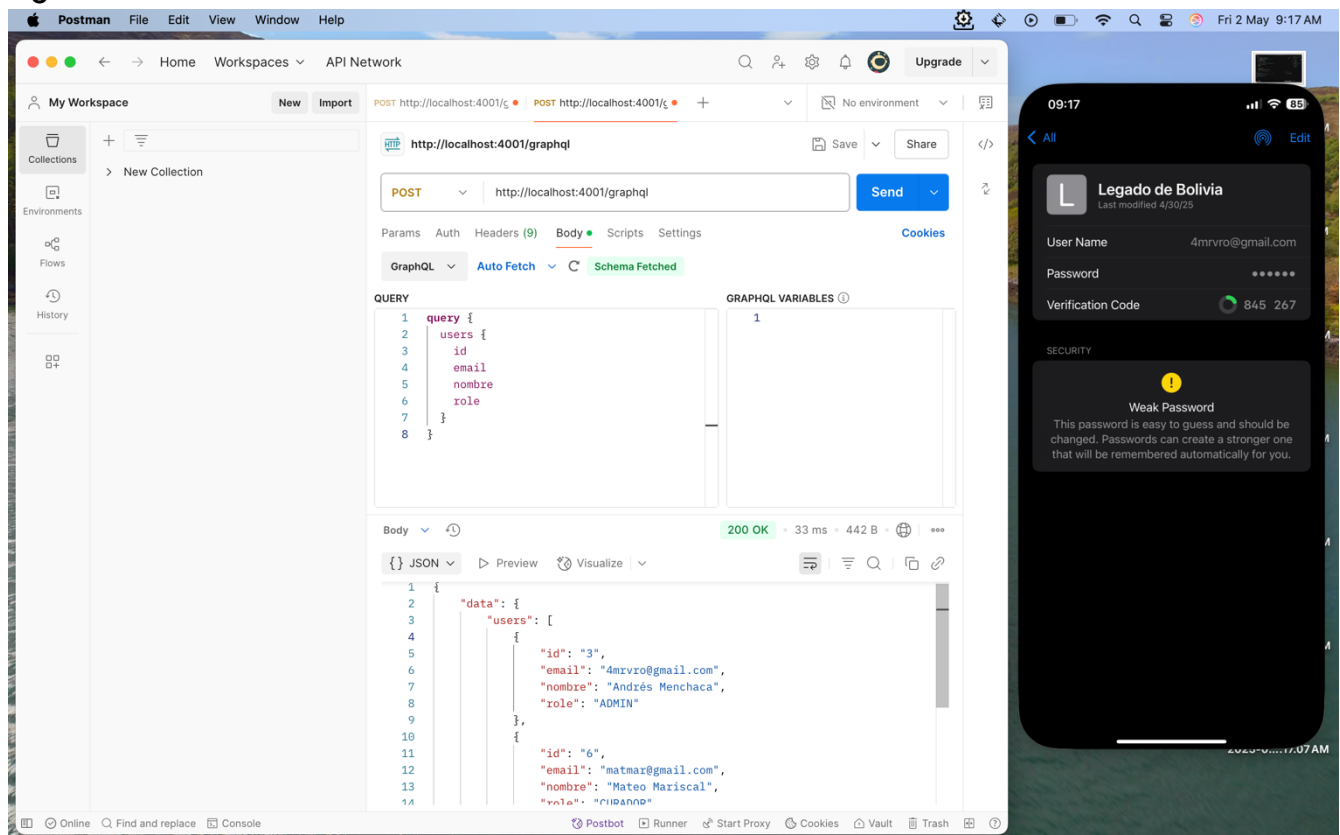
Enviamos la petición POST y nos devolverá el accessToken el cual es el verdadero token para que el ADMIN haga otras consultas que solo él podrá:

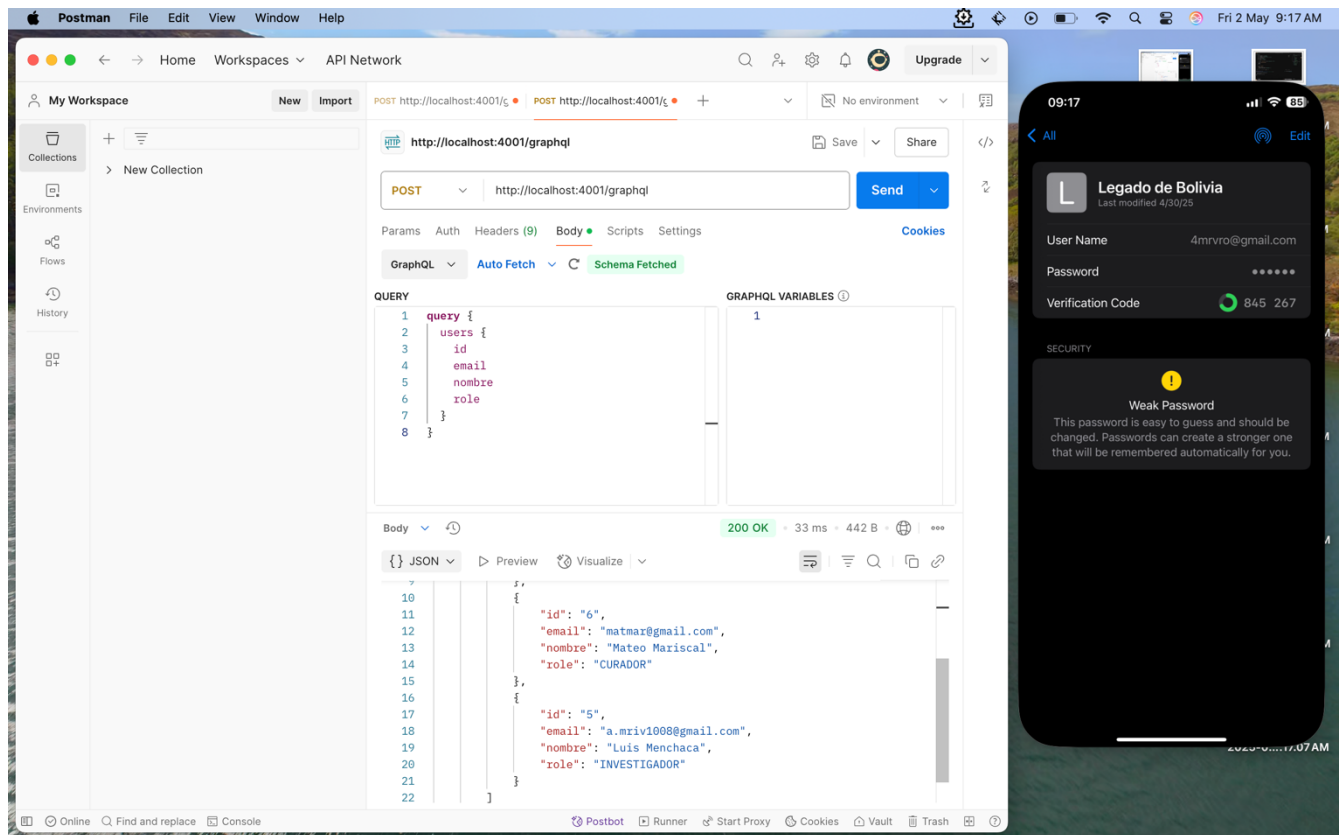


Este accessToken lo copiamos y pegamos en el Header



También cambiamos el Query para obtener los usuarios, enviamos y nos devolverá todos usuarios registrados





```
mutation {
  verify2FA(token: "701926", tempToken: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZi91c3Vhcm1vIjozLCJwZW5kaW5nMkZBIjpb0cnVILCJpYXQi0jE3NDYXOTE3NjEsImV4cCI6MTc0NjE5MjA2MX0.m1kjhFAyciRZ1teyI-dMVkSPowPSPY7WgmMZIpIHLyU") {
    accessToken
    refreshToken
    user {
      id
      email
    }
  }
}
```

3. Obtener usuarios

```
query {  
  users {  
    id  
    email  
    nombre  
    role  
  }  
}
```

- Documentación clara del microservicio (README.md o PDF)

Microservicio en GraphQL para la gestión de usuarios

Microservicio GraphQL que permite a los administradores de la página web de Testimonios consultar usuarios desde la API auth, manejando la autenticación, verificación en dos factores y consultas a través de este endpoint.

Requisitos Previos

- Node.js (versión 18 o superior)
- npm (incluido con Node.js)

Configuración del Entorno

1. Clona el repositorio:

```
```bash  
git clone https://github.com/menchaca-andres/AndresMenchaca_2daEvaluacionBackend.git
cd AndresMenchaca_2daEvaluacionBackend
```
```

2. Instala las dependencias:

```
```bash  
Instalar todas las dependencias
npm install
```

# O si prefieres instalar las dependencias principales manualmente:

```
npm install graphql graphql-yoga @graphql-tools/schema axios dotenv
npm install -D typescript ts-node @types/node @types/graphql
```

```
```env  
PORT=4001  
API_URL=http://localhost:4000  
JWT_SECRET=microservicio-secret-key  
```
```

## ## Estructura del Proyecto

```

...
├─ src/
│ ├── apiClient.ts # Cliente Axios para comunicación con API REST
│ ├── config.ts # Configuración del servidor
│ ├── resolvers.ts # Resolvers de GraphQL
│ ├── schema.graphql # Schema de GraphQL
│ ├── server.ts # Configuración del servidor GraphQL
│ └── types/ # Definiciones de tipos TypeScript
│ └── graphql.d.ts
├─ .env # Variables de entorno
├─ package.json # Dependencias y scripts
└─ tsconfig.json # Configuración de TypeScript
...

```

## ## Scripts Disponibles

- `npm run dev`: Inicia el servidor en modo desarrollo con ts-node
- `npm run build`: Compila el código TypeScript
- `npm start`: Inicia el servidor en modo producción
- `npm run watch`: Inicia el servidor en modo desarrollo con nodemon

## ## Endpoints GraphQL

### ### Queries

- `users`: Obtiene la lista de usuarios (requiere autenticación)

### ### Mutations

- `login(email: String!, password: String!)`: Inicia sesión
- `verify2FA(token: String!, tempToken: String!)`: Verifica el código 2FA

## ## Desarrollo Local

1. El servicio API REST tiene que estar corriendo en `http://localhost:4000`
2. Inicia el servidor en modo desarrollo:

```

```bash
npm run dev
```

```

3. Accede a GraphQL en `http://localhost:4001/graphql` en Potsman con la petición POST para los tres endpoints.

### ## Ejemplo de Uso

#### ### Login

```
``graphql
mutation {
 login(email: "4mrvro@gmail.com", password: "amr2641$") {
 accessToken
 refreshToken
 user {
 id
 email
 }
 requires2FA
 tempToken
 }
}
```

#### ### Verificación 2FA

```
``graphql
mutation {
 verify2FA(token: "701926", tempToken: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZF91c3Vhcm1vIjozLCJwZW5kaW5nMkZBIjp0cnVILCJpYXQiOjE3NDYXOTE3NjEsImV4cCI6MTc0NjE5MjA2MX0.m1kjhFAyciRZ1teyI-dMVkSPowPSPY7WgmMZIpIHLyU") {
 accessToken
 refreshToken
 user {
 id
 email
 }
 }
}
```

#### ### Consulta de Usuarios

```
``graphql
query {
 users {
```

```
 id
 email
 nombre
 role
 }
}
```

Nota: Para la consulta de usuarios, incluye el token JWT en los headers:

```
```.txt
  Authorization  Bearer tu_token
```.
```

## ## Notas Importantes

- El servidor GraphQL corre en el puerto 4001 por defecto
- Se requiere que el servicio API REST esté corriendo en el puerto 4000
- Las peticiones a `/users` requieren un token JWT válido
- El sistema soporta autenticación de dos factores (2FA)